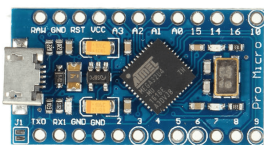
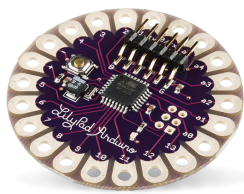


Arduino micro y pro-micro Basado en el microcontrolador **ATmega32u4**, que posee comunicación USB integrada y que facilita el desarrollo de dispositivos de interfaz humana, como teclados, mouse y controladores MIDI. Un clon desarrollado por *SparkFun* (pro-micro) es ampliamente utilizado por el hobby de desarrollo de teclados mecánicos.



Arduino lillypad: Basado en el microcontrolador **ATmega328v** o **ATmega168v**, con el propósito de obtener una placa que facilite la creación de ropa tecnológica. Esta placa está diseñada para ser fácilmente escondida o bordada en ropa que posea capacidades tecnológicas como bio-sensores, luces y otros dispositivos.



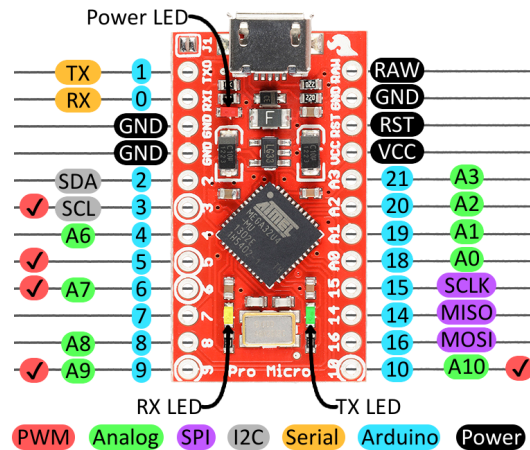
Aparte de estos, existen muchos más, que puedes encontrar en www.arduino.cc/en/hardware.

- **Software:** Entornos de desarrollo, donde se puede escribir programas, administrar librerías y subir instrucciones al hardware compatible. Arduino desarrolló su propio lenguaje de programación basado en **C++** y **Processing**, que facilita el desarrollo de programas enfocados al uso de microcontroladores. Existen dos versiones del entorno de desarrollo, Arduino 1.X y Arduino 2.X. Durante el desarrollo de este club, utilizaremos la versión 2.X, la más actualizada de todas.

Puedes encontrar todas las versiones de software disponibles en www.arduino.cc/en/software.

3.3. Arduino Pro Micro:

Durante la clase pasada ya conocimos a la placa de desarrollo que utilizaremos durante este semestre, el **Arduino Pro Micro**, a continuación veremos las capacidades que tiene:



Esta figura corresponde al **pinout** de la placa de desarrollo, donde se especifican las capacidades de cada una de las entradas y salidas del dispositivo, a continuación, se presenta una breve explicación de cada etiqueta y su color.

- **PWM:** Pines capaces de emitir señales de tipo PWM o "modulación por ancho de pulsos", que permite emular una emisión de señales análogas. Los utilizaremos para controlar servomotores, buzzer piezoeléctricas y cualquier dispositivo que tenga un control análogo.
- **Analog:** Pines con la capacidad de leer señales análogas y transformarlas a un valor digital, los utilizaremos para utilizar sensores que entreguen valores distintos a encendido y apagado (sensores análogos).
- **SPI:** Pines con la capacidad de comunicarse con otros dispositivos mediante el protocolo SPI (no lo veremos durante el taller).
- **I2C:** Pines con la capacidad de comunicarse con otros dispositivos mediante el protocolo I2C, los utilizaremos para controlar una pequeña pantalla OLED.
- **Serial:** Pines que permiten comunicarse con otros dispositivos mediante el protocolo serial, en general utilizamos este protocolo para comunicarnos con el computador mediante USB, pero también puede utilizarse para controlar, y transmitir información entre dos dispositivos.
- **Arduino:** Corresponden a los pines capaces de recibir y emitir señales digitales, serán los pines más utilizados durante el club.
- **Power:** Corresponden a los pines relacionados con la energía eléctrica, que permiten alimentar el arduino desde fuentes externas, distribuir energía a otros dispositivos y reiniciar el procesador de manera eléctrica.

Veamos más en detalle que es lo que significan y como utilizar estos pines.

3.4. Energía:

El arduino pro micro tiene cuatro tipos de pines asociados a energía (etiquetados como **Power** en el pinout), sus funciones y la manera de usarlo son las siguientes:

- **RAW:** Pin mediante el cual se puede alimentar al arduino directamente con un voltaje de entre 5 y 12 [V], en caso de no utilizar el puerto USB para proveerlo de energía.
- **VCC:** Corresponde al terminal positivo de la fuente de energía que la placa de desarrollo es capaz de proveer, sirve para alimentar componentes que necesitan energía de manera constante.
- **GND:** Corresponde al terminal negativo de la fuente de energía o tierra del circuito, todos los componentes y circuitos deben ir conectados a este pin por su terminal negativo.
- **RST:** Pin de reinicio del microcontrolador, cuando este pin es conectado a tierra (**GND**), el procesador se reiniciará.

En resumen, los principales terminales que debemos conocer son **VCC** y **GND**, que actuarán como terminales de una batería para alimentar a ciertos componentes a medida que los utilicemos.

3.5. Señales Digitales:

Las señales digitales son aquellas señales que solo tienen dos estados **encendido** o **apagado**, estos dos estados pueden tener varios nombres:

Encendido	Apagado
HIGH	LOW
ON	OFF
True	False
1	0

El arduino pro micro es capaz de emitir y leer señales digitales a través de sus 18 pines digitales, etiquetados como **Arduino** en el pinout. Veamos que es lo que significan estas dos capacidades, y como realizarlas utilizando funciones predeterminadas:

3.5.1. Lectura de señales digitales:

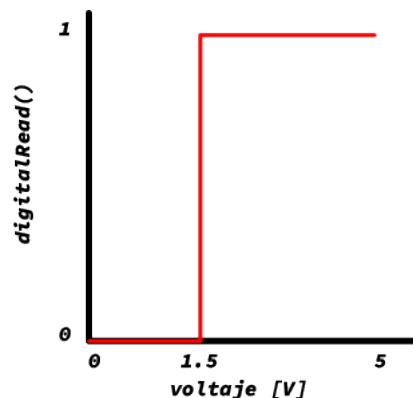
Para configurar unos de los pines digitales como entrada, es necesario utilizar la función de configuración, con el parámetro **INPUT**.

```
1 void setup(){
2   pinMode(pin, INPUT);
3 }
```

Una vez establecido el modo de operación de un pin como entrada digital, podemos obtener el estado que está leyendo utilizando la función :

```
1 int estadoPin = 0;
2
3 void loop(){
4   estadoPin = digitalRead(pin);
5 }
```

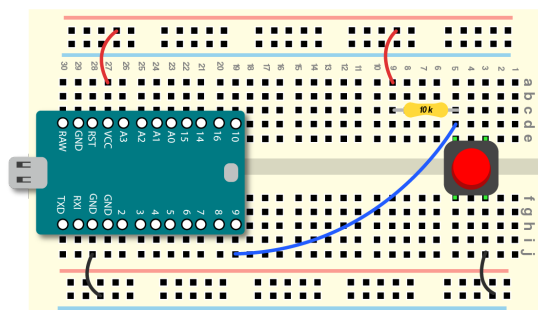
El valor de voltaje que determina cuando la señal está encendida, depende del microprocesador utilizado y su voltaje de operación, en el caso del utilizado por el arduino pro micro es aproximadamente 1.5 [V].



Ejercicio: Vamos a conectar un interruptor directamente al pin, para controlar diversos dispositivos. Para leer la señal proveniente desde este interruptor, podemos conectarlo de dos maneras:

- **PULLUP:** Conectamos VCC al pin que utilizaremos mediante una resistencia de 10K. Conectamos VCC a GND mediante el interruptor.

De esta manera, el voltaje proveniente de la fuente de poder siempre está conectada al pin mediante la resistencia y cuando apretamos el interruptor, el voltaje bajará en el pin, ya que el camino hacia el terminal negativo del arduino (GND) ofrece menos resistencia.

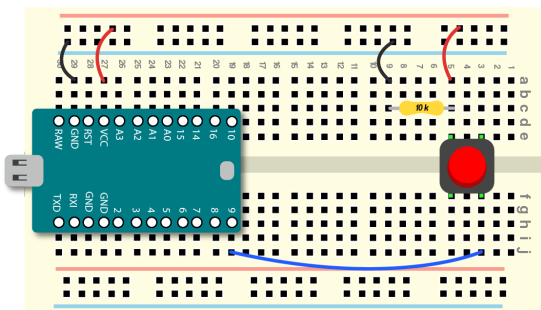


Cuando conectamos el interruptor de esta manera, obtendremos los siguientes resultados de **digitalRead()**:

Interruptor	<i>digitalRead()</i>
OFF	1
ON	0

- **PULLDOWN:** Conectamos VCC a GND mediante una resistencia de 10K. Conectamos VCC al pin que utilizaremos mediante el interruptor.

De esta manera, el voltaje proveniente de la fuente de poder siempre está conectada al terminal negativo (GND) mediante la resistencia y cuando apretamos el interruptor, el pin observará un voltaje positivo, ya que el camino hacia este ofrece una menor resistencia.



Cuando conectamos el interruptor de esta manera, obtendremos los siguientes resultados de `digitalRead()`:

Interruptor	<code>digitalRead()</code>
OFF	0
ON	1

Ahora prueba estos dos circuitos, y comprueba el comportamiento que acabamos de mencionar, utilizando el **monitor serial**.

3.5.2. Emisión de señales digitales:

Para configurar unos de los pines digitales como salida, es necesario utilizar la función de configuración, con el parámetro `OUTPUT`.

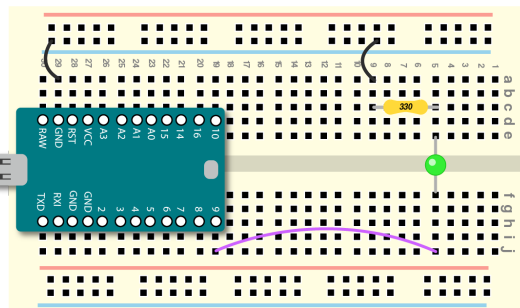
```
1 void setup(){
2   pinMode(pin, OUTPUT);
3 }
```

Una vez configurado, el pin podrá ser utilizado como una fuente emisora de señal digital, es decir, se podrá controlar de manera precisa cuando se enciende y se apaga el voltaje desde ese pin, con esto podremos controlar el encendido y apagado de cualquier componente digital, que requiera una señal de 5v y podremos comunicarnos con dispositivos que tengan algún tipo de comunicación por señales digitales (como múltiples tipos de sensores y pantallas).

Utilizando la función `digitalWrite()`, podrás encender o apagar dispositivo que requieran de un voltaje constante.

```
1 void loop(){
2   // Encender el pin digital
3   digitalWrite(pin, HIGH);
4   // Apagar el pin digital
5   digitalWrite(pin, LOW);
6 }
```

Ejercicio: Ejercitemos el uso de esta función implementando el ejercicio más simple **BLINK**, conectemos un LED y una resistencia como se ve en el diagrama, y completen los siguientes desafíos:



- **1) Blink Básico:** Hacer parpadear la luz con un intervalo de 1 segundo.
- **2) SOS:** Hacer parpadear el LED con un patrón de SOS: tres parpadeos cortos, una pausa corta, tres parpadeos largos, una pausa corta, tres parpadeos cortos y una pausa larga (.../—/...).
- **3) Acelerando:** Hacer parpadear la luz cada vez más rápido, utilizando un *ciclo for* o un *ciclo while*, partiendo con la luz encendida por 2 segundos acortando el tiempo cada vez que se apaga.

Repetamos este ejercicio, pero utilizando el buzzer activo.



Ejercicio: Ahora, mezclemos las dos capacidades (input y output digital) para controlar distintos LEDs. Para facilitar el desarrollo de este ejercicio y disminuir la cantidad de componentes utilizados, revisemos la siguiente información.

Existe también el modo **INPUT_PULLUP** en arduino, que activa un circuito de tipo pullup, conectado directamente al pin digital, ahorrando la necesidad de armar el circuito de manera externa.

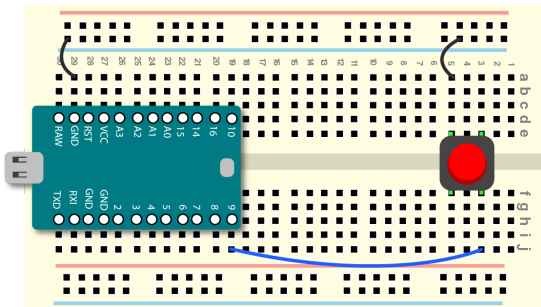
Simplemente conecta un terminal del interruptor al pin de la placa, y otro terminal a tierra, y podrás utilizarlo tal y como si hubieras hecho el circuito que probamos en el ejercicio anterior. Procura utilizar la siguiente línea en el `setup()`:

```

1 void setup(){
2   pinMode(pin, INPUT_PULLUP);
3 }

```

y conecta de la siguiente manera:



Sabiendo esto, intenta resolver los siguientes problemas:

1. **ON/OFF:** Utiliza botón para encender y apagar una luz LED, de la siguientes dos maneras:
 - La luz LED deberá encenderse solo cuando el botón está apretado
 - La luz LED deberá encenderse cuando se apriete el botón y deberá apagarse cuando se vuelva a apretar el botón.
2. **Semaforo:** Conecta tres LEDs a tres pines distintos, e intenta simular un semáforo, haciendo que el LED que esté encendido, cambie cada vez que se aprieta el botón.
3. **Contador:** Con los mismos tres LEDs, intenta implementar un contador, que vaya aumentando la cantidad de luces encendidas cada vez que aprietas el botón, una vez que llegues a 4, apáguelas todas.
4. **Contador Binario:** Con los mismos tres LEDs, implementa un contador binario, que encienda y apague las luces representando la cantidad de veces que se haya apretado el botón con un número binario, como se ve en los siguientes ejemplos:

Veces Apretado	LED 1	LED 2	LED 3
0 = 000	OFF	OFF	OFF
1 = 001	OFF	OFF	ON
2 = 010	OFF	ON	OFF
3 = 011	OFF	ON	ON
4 = 100	ON	OFF	OFF
5 = 101	ON	OFF	ON
6 = 110	ON	ON	OFF
7 = 111	ON	ON	ON

3.6. Señales Análogas:

Las señales análogas, son aquellas señales que pueden tomar cualquier valor posible, y no están limitadas a encendido y apagado, en el caso de la electricidad, correspondería al valor del voltaje de la señal. Las placas de desarrollo arduino poseen un componente llamado “**Convertor Análogo a Digital (ADC)**”, que permite leer valores análogos desde componentes electrónicos, y almacenar los como un valor numérico. La precisión de estos ADC dependen de la cantidad de bits en la que pueden dividir la señal.

3.6.1. Lectura de señales análogas:

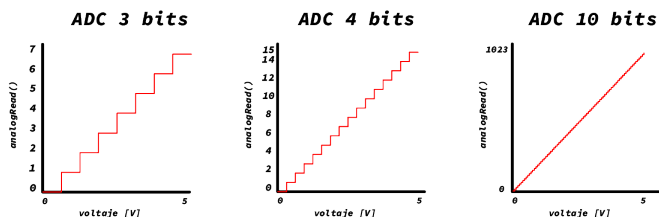
Para leer señales análogas desde un pin, no es necesario configurarlo en el `setup()`, sólo se requiere que el pin sea de tipo análogo, es decir, que esté etiquetado como [AX]. La lectura se realiza utilizando la siguiente función:

```

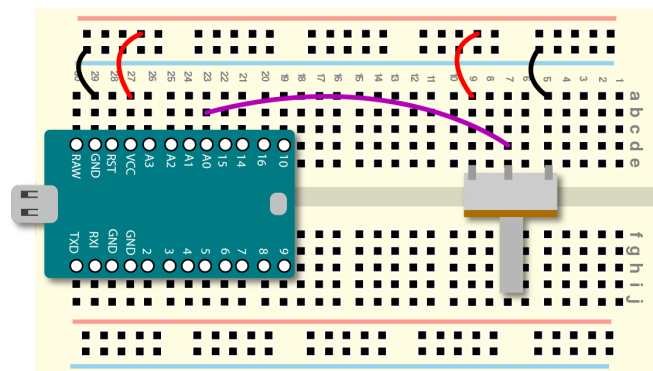
1 int estadoPin;
2
3 void loop(){
4   estadoPin = analogRead(AX);
5 }

```

Esto entregará un valor entre 0 y 1023, ya que nuestro arduino posee un ADC (Convertor analógico a digital) de 10 bits. Este convertor funciona midiendo el voltaje recibido y transformándolo a un número que podemos interpretar en nuestro código. Mientras más bits tiene, se tendrá una mayor resolución a la hora de medir el voltaje que recibe el pin, tal y como se muestra en la siguiente figura:



Ejercicio: Observemos como funciona la lectura de señales análogas, conectando una resistencia variable (potenciómetro) y viendo los valores que aparecen en la pantalla. Sigamos el siguiente diagrama:



Observemos los valores que arduino lee utilizando el **monitor serial**.

Ejercicio: Utilicemos el potenciómetro para controlar el comportamiento de uno o varios LEDs

1. **Blink 2:** Utiliza los valores del potenciómetro para controlar la velocidad con la parpadea un LED.
2. **Indicador:** Utiliza el potenciómetro para controlar la cantidad de LEDs que se encienden a medida que lo giras, parte con 2 y ve si puedes llegar a 4. *Este desafío necesita que investigues el uso de la función `map()`, queda como parte del ejercicio buscar cómo funciona en internet y descubrir por qué sirve para solucionar esto! discútelo con tus compañeros y pide ayuda al mentor.*

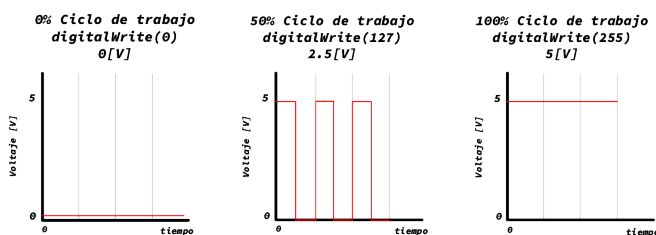
3.6.2. Escritura de señales analógicas:

La escritura de señales analógicas se realiza utilizando una técnica llamada **PWM** (*Pulse Width Modulation*, o *modulación por ancho de pulsos*).

Para emitir una señal analógica, no es necesario configurar el modo del pin en el setup, solo debe asegurarse que el componente al que se le enviará el voltaje esté conectado a alguno de los pines que pueden emitir PWM.

```
1 void loop(){
2   // valor = 0 -> 255
3   analogWrite(pinPWM, valor);
4 }
```

Los valores que arduino puede emitir van de 0 a 5 V, en 256 intervalos, es decir, puedes ir aumentando el voltaje en incrementos de 0.02 [V] aproximadamente.



Ejercicio: Utilicemos la función `analogWrite()` para controlar el brillo de un LED, a partir de la posición de la perilla de un potenciómetro. Utiliza los diagramas de los ejercicios anteriores para hacerte una idea de cómo conectar los componentes.

3.6.3. Otros usos de PWM

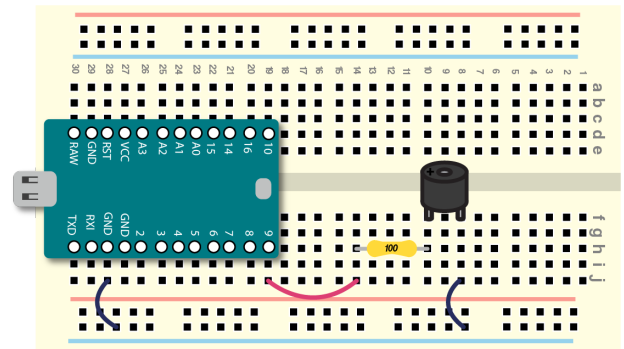
Existen distintas funciones y librerías que utilizan la capacidad de Arduino de modular pulsos, para controlar distintos tipos de dispositivos que pueden ser controlados o activados al variar el voltaje que se les entrega, podemos ver dos ejemplos simples: **buzzer piezoeléctrico pasivo** y **servomotores**.

3.6.4. buzzer piezoeléctrico pasivo

Podemos utilizar la función `tone()`, para generar un tono con alguna frecuencia, idealmente entre 200 y 5000 [Hz], para generar sonidos utilizando un buzzer piezoeléctrico pasivo. La manera en la que se utiliza esta función es la siguiente:

```
1 // Definir el pin
2 const int piezo = 9;
3
4 // Configurar el pin como output
5 void setup(){
6   pinMode(piezo, OUTPUT);
7 }
8
9 void loop(){
10  // Enviar un tono
11  tone(piezo, frecuencia);
12  // Apagar el tono
13  noTone(piezo);
14 }
```

El buzzer debe estar conectado de la siguiente manera:



Ejercicio: Tenemos los siguientes desafíos que involucran al buzzer piezoeléctrico.

1. **Canción:** Escribe una breve canción utilizando tonos de distintas frecuencias y delays, puedes hacerlo al oído o buscando las notas en internet. Algo útil es tener una lista de frecuencias y sus notas correspondientes, como por ejemplo la siguiente calculadora <https://www.translatorscafe.com/unit-converter/es-ES/calculator/note-frequency/>
2. **Instrumento:** Controla la nota emitida por el buzzer piezoeléctrico utilizando un potenciómetro.
3. **Instrumento 2:** Agrega un botón para que el piezo solo suene cuando se aprieta el interruptor.
4. **Instrumento 3:** transforma el sonido para que emita un arpeggio (investigar que es un arpeggio) en vez de una nota. Ahora estaríamos acercándonos a tener un sintetizador.